



Efficient tools with **MCP Code Mode**

How Code Mode makes large MCP tool surfaces faster, cheaper, and more practical for agents



The promise of MCP

Model Context Protocol gives agents a standard way to use external tools.

- Search docs

- Query APIs

- Read databases

- Create tickets

- Manage cloud infrastructure

- Operate internal systems



**MCP turns models from chat
interfaces into software
operators.**



The problem: tools are **not free**

In traditional MCP, every capability is exposed as a separate tool. Each tool usually brings:

- **A name**
- **A description**
- **An input schema**
- **Parameter descriptions**
- **Return shape expectations**

TOOL DEFINITION

```
{  
  "name": "create_ticket",  
  "description": "Creates a...",  
  "inputSchema": {  
    "type": "object",  
    "properties": { ... }  
  },  
  "returns": { ... }  
}
```

All of this fits in context before work even starts.



Tool surfaces grow quickly

Small MCP servers are easy. Large platforms are different.

- **Cloud provider** — Thousands of API endpoints
- **Enterprise portal** — Dozens of internal MCP servers
- **Multi-tool workflow** — Jira + Google Drive + GitHub + Slack + databases + internal services



Native MCP at scale

Direct tool exposure creates three kinds of pressure.

1

Token cost

Schemas consume context and money. Every tool definition eats into the window before the agent can even start working.

2

Latency

Each tool call often requires another model round trip. Multi-step workflows multiply the delay.

3

Reliability

Many similar tools make selection harder. The model may pick the wrong tool or hallucinate parameters.

The result: more tools, but less room to think.



Code Mode in one sentence

Code Mode replaces a large list of direct tool definitions with a **small code interface.**

Instead of asking the model to call many tools one at a time, the model writes JavaScript or TypeScript that searches, composes, and executes tool calls inside a sandbox.

Many tools, one at a time



Small code interface



The mental model

Two approaches to multi-step tool use.

SEQUENTIAL

Traditional MCP

STEP 1

Model chooses a tool

STEP 2

Calls it, reads the result

STEP 3

Chooses another tool

STEP 4

Calls it again

PROGRAMMATIC

Code Mode

STEP 1

Model writes a small program

STEP 2

Program calls tools, handles data

STEP 3

Returns only the useful result



Why code helps models

LLMs have seen a huge amount of real-world code. They are often better at writing code against typed APIs than selecting from a large menu of synthetic tool schemas.

FAMILIAR CONTROL FLOW

- Filtering

- Mapping

- Pagination

- Retries

- Branching

- Error handling

- Data shaping

CODE MODE

```
const results = await
  api.search("DNS zones")

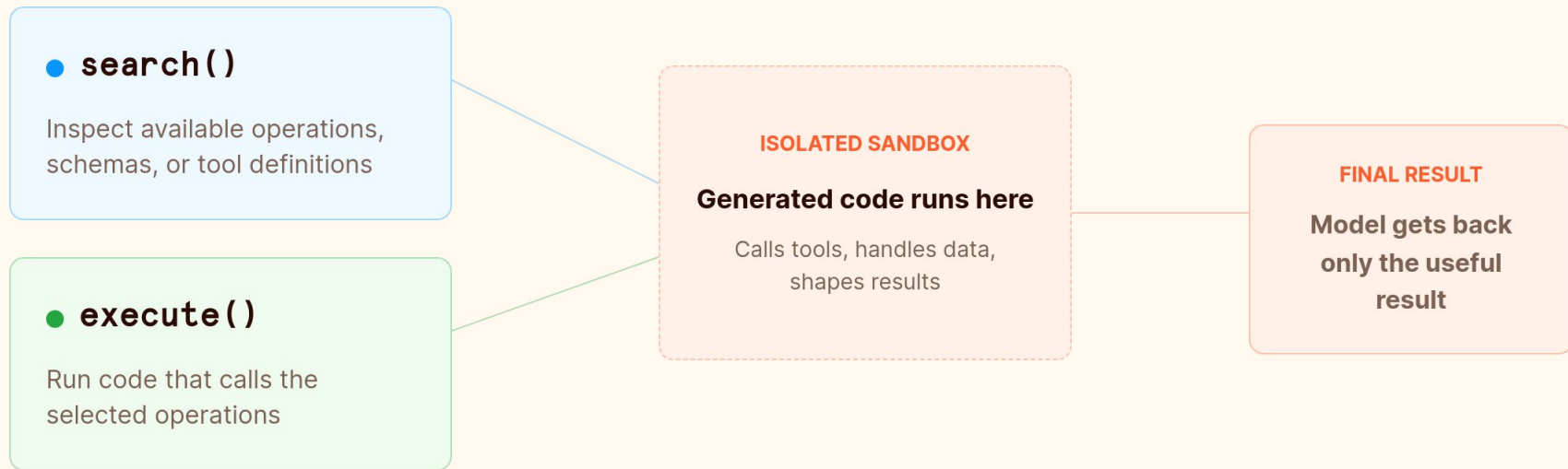
const active = results
  .filter(z => z.status === "active")
  .map(z => z.name)

return active
```




How Code Mode works

A Code Mode MCP server usually exposes a small number of tools.



EXAMPLE



Cloudflare API: 2,500+ endpoints

Exposing every endpoint as a native MCP tool would require a huge amount of context.

WITH CODE MODE — JUST 2 TOOLS

- **search()**

Discover the right endpoint

- **execute()**

Call the API

~1,000

tokens for the entire API

vs. native full schemas

~1.17M tokens



Why this matters for tokens

The key improvement is fixed tool overhead. Native MCP grows with the number of tools. Code Mode stays small.

2,500+

Cloudflare API endpoints in ~1,000 tokens

Code Mode discovers capabilities on demand

~1.17M

Tokens for native full schemas

Every endpoint exposed as a separate tool definition

94%

Token reduction for portal tools

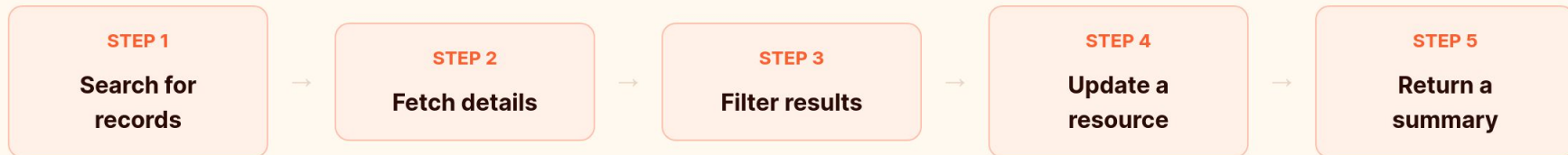
52 tools: ~9,400 tokens → ~600 tokens





Why this matters for **speed**

Native tool calling often requires multiple model round trips. Code Mode can combine steps into one execution.



NATIVE MCP

5 steps = 5 round trips

Model reads and re-emits every intermediate value

CODE MODE

5 steps = 1 execution

Code handles all intermediate data internally



Why this matters for **composition**

Real workflows rarely use one tool. They combine systems.

1

- Find a Jira ticket

2

- Pull context from Google Drive

3

- Check related GitHub issues

4

- Update the ticket

5

- Send a Slack summary

CODE MODE ADVANTAGE

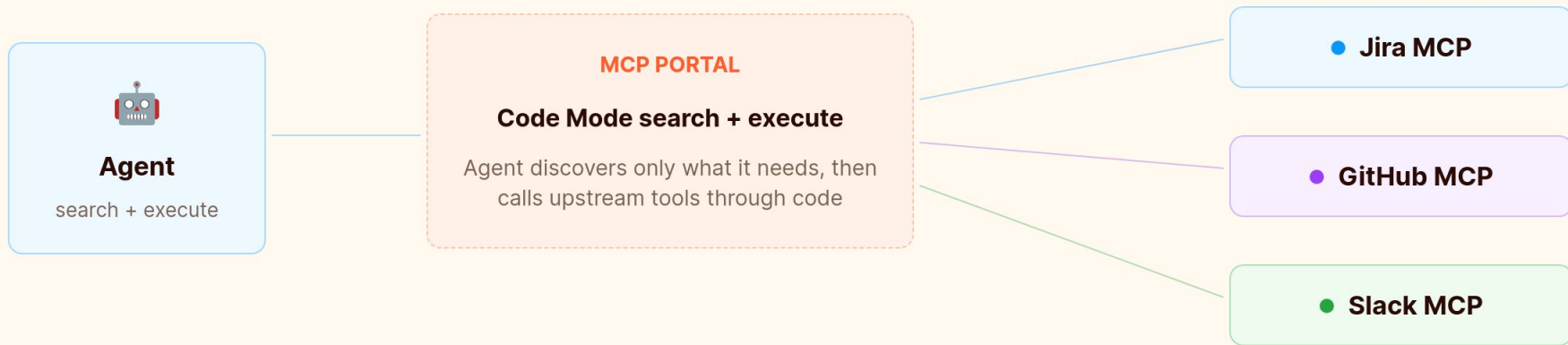
Code Mode lets the agent compose these actions using normal programming constructs.

```
const ticket = await jira.find(id)
const docs = await drive.search(ticket)
const issues = await github.related(ticket)
await jira.update(ticket, { docs, issues })
await slack.notify(summary)
```



Code Mode for MCP Portals

MCP Server Portals let organizations put multiple MCP servers behind one governed endpoint.



DEMO



●
Live Demo



- **Code Mode docs**

<https://developers.cloudflare.com/agents/api-reference/codemode/>



When Code Mode is a **good fit**

Use Code Mode when:

- **The tool surface is large**
- **Workflows require multiple tool calls**
- **Intermediate data is noisy or large**
- **The agent needs to search capabilities dynamically**
- **Token cost or latency matters**
- **You need safer secret handling around generated code**

NOT ALWAYS NEEDED

Small, simple MCP servers with a handful of well-defined tools may not need Code Mode.



Thank you.

@megaconfidence



